Summary

# Architecting AI with MCP Session

## Overview

In this session, Shubhangi (Technical Lead, Microsoft MVP) outlines the architectural principles for implementing the **Model Context Protocol (MCP)** within enterprise AI environments. MCP is framed as the "USB for AI," providing a standardized, auditable, and composable layer to connect LLMs with disparate data sources (e.g., SQL Server, GitHub, Slack, and Guidewire) without building bespoke APIs for every integration.

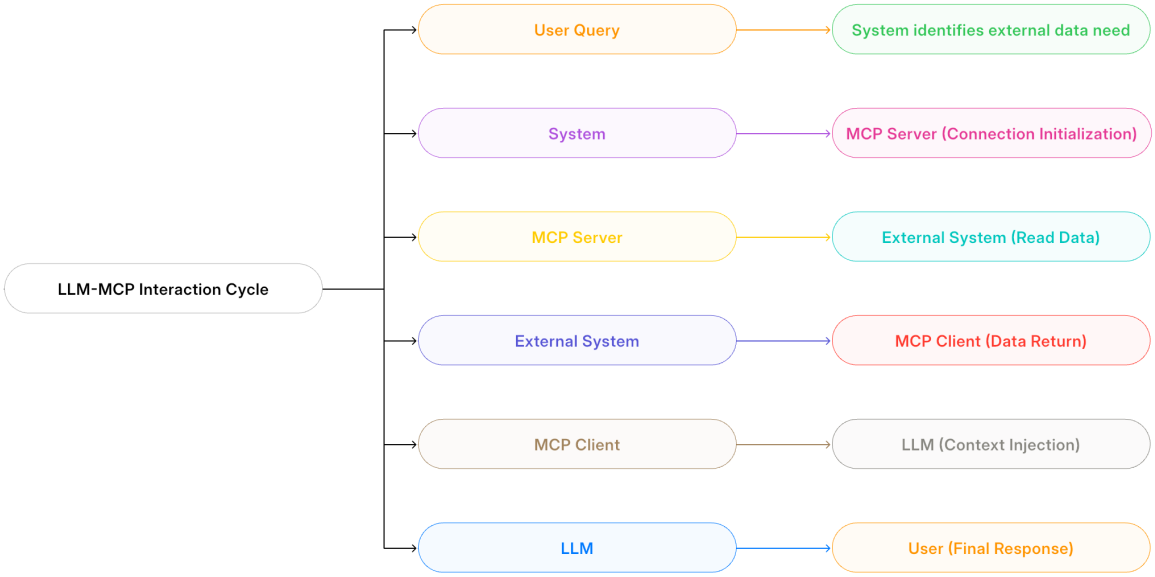
## Core Architecture & Interaction Flow

The MCP architecture transitions from fragmented, point-to-point integrations to a unified API model.

## Standard Components:

- **Host:** The application utilizing the LLM (e.g., VS Code, Cloud Desktop).
- **MCP Client:** The interface within the host application that manages the protocol.
- **MCP Server:** The source-specific connector providing context and capabilities (e.g., Postgres, GitHub).
- **Transport Layer:** The communication method between client and server, typically utilizing HTTP/IO or HTTP/2.

### LLM-MCP Interaction Cycle




## Operational Deployment: Foundry MCP

Successful implementation on platforms like Microsoft Azure Foundry requires a specific configuration of environment variables and JSON schemas.

## Prerequisites for Implementation:

- Active Azure subscription and a pre-configured Foundry project.
- VS Code with GitHub Copilot extension.
- Secure environment variable file (referenced in `mcp.json`) to store API keys and endpoints.

## Common Triage Points:

- **Error 401:** Missing endpoint authentication.
- **Pathing Failures:** Use of relative rather than absolute full paths in JSON configurations.
- **Platform Broker Errors:** Occurs when Windows brokers are disabled; requires a specific platform call to resolve.
- **Cache Bloat:** Frequent in Azure Search integrations; requires a cache refresh/re-work.

## Strategic Patterns for Reliable Deployment

To move from PoC to production, four patterns were emphasized:

1. **Connection Pooling:** Maintain persistent server connections rather than spinning them up per request to minimize latency.

2. **Async Tool Execution:** For long-running jobs, return a Job ID and use polling results.
3. **TTL-Based Caching:** Cache deterministic tool outputs to reduce API costs and latency.
4. **Observability:** Implement structured logging to trace every tool call's inputs, outputs, and latency.

## Context Management & Security

The session concluded with a warning against "Context Chaos." Simply giving an agent data access does not equate to intelligence; it requires rigorous engineering.

- **Memory Management:** Short-term (working) memory often resides in dictionaries/objects within the MCP server. For multi-agent architectures, focus on passing session information between agents to prevent the "blank slate" experience for users.
- **Data Hygiene:** Filter and summarize data before feeding it to the model. In one cited example, 65% of an enterprise knowledge base was found to be redundant/unused.
- **Security Protocol:** Validate all inputs and sanitize outputs. Use a **Principle of Least Privilege**—do not grant high-level data access to agents by default.
- **Risk Classification:** Classify data into High, Medium, and Low risk tiers at the start of the process to ensure PII is redacted before entering the context window.

## Action Items

- **Test Connection:** Use prompts like "Tell me about latest models on Foundry" to verify that the MCP server connection is established and permissions are active.
- **WebNCP Review:** Evaluate the existing WebNCP architecture against the recommended connection pooling and TTL-based caching patterns to optimize performance.
- **Logging Update:** Ensure structured logs are implemented to capture tool call depth and token length to prevent context window stalls.